ORGANIZATIONAL REVIEW

TaskManage

Project and task management for teams that need a shared view of ownership, deadlines, and progress.

Project evaluation report

This report describes the application as implemented in the submitted repository. It summarizes its main workflows, access rules, technical design, test evidence, and remaining work against the supplied requirements.

AT A GLANCE	DETAIL
Project type	Web application for project, team, and task coordination
Primary roles	Administrator, project manager, team member
Data store	PostgreSQL with versioned SQL migrations
Evaluation snapshot	Source review plus 4 passing Jest integration tests and a successful production build
Deployment	No live URL is included in the repository

Purpose

TaskManage gives a project team one place to define project ownership, break work into assigned tasks, record discussion, and check progress. The dashboard is scoped to projects visible to the signed-in user; administrators can review organization-wide activity.

What an evaluator can review

- A role-aware interface for project, task, and account workflows.
- A REST API backed by relational data, with project membership checks on protected operations.
- Database schema, migrations, API reference, ER diagram, and a Postman collection.
- Automated integration coverage for authentication, permissions, project/task workflows, files, dashboards, and password reset.

How the application works

A user signs in and receives a bearer token. The API loads the account role from the database and checks project ownership or membership before returning protected project data. The browser provides the interface; the server owns validation and authorization decisions.

Roles and responsibilities

Role	Current capabilities
Administrator	Manage user accounts and roles, assign project managers, review audit logs, and perform administrative project/task actions.
Project manager	Create and manage owned projects, add or invite project members, organize tasks, and review project progress.
Team member	View projects they belong to, update assigned tasks, comment on tasks, and upload or download project files.

Typical workflow

Step	Action	Result
1	A manager creates a project and sets an optional deadline.	The manager owns the project; its data is stored in PostgreSQL.
2	The manager adds an existing account or creates an invitation.	Accepted invitees become project members; invitations expire after seven days.
3	The team creates tasks and sets assignees, priority, status, and due date.	Task edits are checked against project membership and role.
4	Members discuss tasks and share attachments.	Comments, file metadata, and status history are linked to tasks.
5	The team reviews the dashboard.	Progress, task totals, team assignments, and open deadlines are summarized.

Data and service design

The Express API exposes JSON endpoints for accounts, projects, team membership, tasks, comments, attachments, audit records, and dashboards. PostgreSQL stores users, projects, memberships, invitations, tasks, status history, comments, attachments, reset tokens, and audit logs. Uploaded file contents are stored on disk in the configured upload directory; the database stores their metadata.

The browser client is a React application. The repository also includes Dockerfiles and Docker Compose configuration. Database setup is handled by the base schema and ordered migrations.

Requirements coverage

The supplied requirements are broader than the current implementation. This table distinguishes a working feature from a partial match and from a requirement that is not currently present.

Requirement area	Status	Implementation evidence / gap
Registration, login, JWT	Implemented	Member-only public signup, password hashing, bearer-token authentication, and protected routes.
Password recovery	Implemented	Single-use expiring tokens; email delivery depends on configured Resend credentials.
Role-based access	Implemented	Admin, manager, and member roles; project-specific ownership and membership checks.
Project CRUD, ownership, search	Implemented	Project create/edit/delete, manager assignment, status filter, search, and pagination.
Team membership and invitations	Implemented	Manager-only add/invite controls; API rejects non-managers. Invitations are accepted by the invited signed-in email.
Task tracking	Implemented	Task create/edit/delete, assignees, three priorities, three statuses, dates, filters, sorting, and history.
Comments and collaboration	Partial	Comments are stored and refreshed by polling while the task panel is open; this is not push-based real-time delivery.
File attachments	Implemented	Upload, list, download, and deletion with a 5 MB application limit; deployment needs persistent storage.
Progress and deadline overview	Implemented	Scoped progress totals, team task counts, and up to ten open tasks with due dates.
Deadline notifications	Not implemented	The dashboard displays upcoming/overdue work; there is no notification or alert delivery workflow.
Performance metrics	Partial	Team view reports task counts and completed counts; it is not a broader productivity or performance measurement system.
Validation and audit logs	Implemented	API input checks, generic client errors, and logged user actions for key workflows.
API documentation and ERD	Implemented	Repository contains endpoint reference, ER diagram, schema, migrations, and Postman collection.
Automated testing	Partial	Four database-backed Jest integration tests pass. The repository does not establish broad unit-test coverage or a coverage target.
Docker support	Implemented	Dockerfiles and Compose configuration are present; deployment was not executed for this review.
Live deployment	Not provided	No application URL or production hosting evidence is present in the repository.

Status reflects source review and the local verification described in this report. It is not a production security or load certification.

Quality and verification

Verification performed

- Jest integration suite: 4 tests passed, covering validation and unauthenticated access; project, membership, task, comment, attachment, audit, and dashboard workflows; and password recovery.
- React production build: completed successfully.
- The integration suite used a configured database and temporary test accounts. The run required database network access; the earlier sandboxed run could not connect.

Security and data handling

- Public account registration cannot grant elevated roles. Initial administrator setup is a trusted database operation.
- Passwords are stored as bcrypt hashes; password reset and invitation tokens are stored as hashes and have expiry checks.
- API authorization checks are backed by server-side role, project-owner, and membership checks; hiding a UI control is not the only access control.
- API error responses avoid returning SQL details to clients. Rate limits apply to login and password recovery requests.
- File uploads are size-limited and use generated storage names. A production host must mount persistent storage and protect deployment secrets.

Known gaps and deployment notes

- No notification service sends deadline alerts. Users must check the dashboard for approaching or overdue tasks.
- Task discussion refreshes on a five-second polling interval while open. No WebSocket or server-sent event channel is implemented.
- Password recovery and email invitations require an email provider configuration. Without it, local development can return a manual token or invitation link.
- The repository has Docker configuration but no live deployment URL. A hosted evaluation needs a provisioned database, secrets, email settings if required, and persistent upload storage.
- The test suite covers primary workflows but does not report a formal code-coverage percentage or load-test results.

Evaluation outcome

The repository demonstrates a working project and task management application with database-backed access control and end-to-end API workflow coverage. The main items to address before production use are deadline notifications, deployment, email configuration, persistent file storage, and broader automated quality coverage.

Review and run guide

Suggested reviewer walkthrough

Order	Review action	What to observe
1	Register an account and sign in.	New public accounts receive the member role.
2	Promote a first administrator through the trusted database console, then sign in as that account.	The Admin area can manage roles and assign project managers.
3	Create a project and add a team member.	The project owner can manage the project and add members or invite them.
4	Sign in as a non-manager and try the same membership actions.	The add/invite controls are hidden and the API returns forbidden for direct requests.
5	Create tasks, assign them, change status, add a comment, and attach a small file.	Check task status history, discussion, file download, and project visibility.
6	Review the dashboard and audit log as the appropriate role.	Dashboard data follows project access; audit records are admin-only.

Local evaluation setup

Requires Node.js 20+, npm, and a reachable PostgreSQL database. Install root and client dependencies with npm ci, provide DATABASE_URL and a high-entropy JWT_SECRET to the API, apply migrations with npm run migrate, then run the API and React client in separate terminals. README.md contains the full local and Docker instructions.

For password recovery and email invitations, configure RESEND_API_KEY and EMAIL_FROM. Configure CLIENT_ORIGIN for the browser URL. In hosted environments, use a persistent UPLOAD_DIR. Keep all secrets in the host's secret manager.

Repository materials

Resource	Location
Setup and operations	README.md
Requirements source	EXTRACTED_REQUIREMENTS.md
Endpoint reference	documentation/API.md
Database design	documentation/ERD.md and server/models/schema.sql
Database updates	server/migrations/
API request examples	postman/TaskManage.postman_collection.json
Integration tests	tests/app.test.js

This report describes the submitted source tree. Production hosting, email delivery, and runtime configuration must be verified in the target organization environment.